M2103/TP 5 - Exceptions

M Billaud

07-03-2021

Table des matières

1 Création d'un projet **1**

2 Étape 1 : établissement et groupes **1**

3 Étape 2 : problème avec les doublons. **2**

4 Utiliser les exceptions (suite) **3**

5 Groupes et étudiants **3**

6 Regroupement des exceptions **3**

7 Groupes et profs **4**

Objectifs :

On va implémenter partiellement une structure de données qui contient les données pour un **établissement**‘ d'enseignement, avec des **élèves** qui sont dans des **groupes**, et des suivent des **cours* avec des **enseignants**.

Ça sera l'occasion d'utiliser

- divers conteneurs (sets, maps …) pour représenter des associations,
- l'héritage (les élèves et les enseignants sont des **personnes**)
- des exceptions pour signaler les anomalies,
- les modificateurs d'accès.

1 Création d'un projet

- Créez un nouveau projet Netbeans
- déclarez un package `iutbx.votreidentifiant.school`, qui vous servira pour les classes qui représentent les données de l'école. Dans ce qui suit, on l'appellera “le package”.

2 Étape 1 : établissement et groupes

Chaque groupe possède un nom (exemple “S2B”), et d'autres attributs (dont les élèves, on verra plus tard)

```
public class Group {
    final String name;
    // autres attributs à venir

    Group(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
```

```
    }
}
```

La visibilité du constructeur est volontairement limitée au package. Les groupes sont créés par des instances de `School`, pas par du code “extérieur”.

Les groupes d’un établissement ont des noms distincts. Pour les retrouver facilement, ils sont rangés dans un tableau associatif (**map**) indexé par le nom.

```
public class School {

    final String schoolName;

    // Groupes, indexés par leur nom
    final Map<String, Group> groups = new TreeMap<>();

    public School(String schoolName) {
        this.schoolName = schoolName;
    }

    /**
     * Crée et enregistre un nouveau groupe
     * @param groupName  nom du groupe
     * @return           le groupe
     */
    public Group registerGroup(String groupName) {
        Group g = new Group(groupName);
        groups.put(groupName, g);
        return g;
    }

    /**
     * Retourne les groupes enregistrés.
     * @return une collection
     */
    public Collection<Group> allGroups() {
        return groups.values();
    }

    /**
     * Cherche un groupe par son nom.
     * @param le nom du groupe cherché
     * @return le groupe si présent, null si absent.
     */
    public Group groupByName(String name) {
        return groups.get(name);
    }

}
```

Travail demandé :

1. Intégrez ce code dans le package.
2. Dans la méthode `main` du projet, définissez et appelez une méthode `test1` qui crée une école, y ajoute 3 groupes, et fait afficher la liste des groupes.

```
School s = new School("BigSchool");
s.registerGroup("S2A");
s.registerGroup("S2B");
s.registerGroup("S2C"); // oups
for (Groupe g : s.allGroups()) {
```

```
    System.out.println(get.getNom());
}
```

3 Étape 2 : problème avec les doublons.

1. **Écrivez une méthode** `test2` qui ajoute deux groupes portant accidentellement le même nom, et les fait lister.

Vous constatez que le résultat n'est pas satisfaisant.

- au niveau de `School`, on aurait dû détecter que le nom existait déjà.
- mais c'est le code de `test2` qui décide des mesures à prendre.

2. Une possibilité, abandonner ce test si quelque chose se passe mal, en englobant les instructions dans un `try-catch`

```
try {
   School s = ....
   ...
} catch (GroupRegistrationException e) {
   System.out.println("Erreur enregistrement : " + e.getMessage());
}
```

3. Déclarez l'exception dans le package

```
public class GroupRegistrationException extends Exception {
    public GroupRegistrationException(String message) {
        super(message);
    }
}
```

4. Dans la méthode `registerGroup`, détectez la situation problématique, et signalez-la :

```
if (groups.containsKey(name)) {
    throw new GroupRegistrationException(
         "Group " + name + " already present");
}
```

Que faut-il ajouter à la signature ?

4 Utiliser les exceptions (suite)

On décide de faire autrement. Dans `test3`, mettez une copie modifiée de `test2`

```
School s = new School("BigSchool");
addIfPossible(s, "S2B");
addIfPossible(s, "S2A");
addIfPossible(s, "S2B");
for (Groupe g : s.allGroups()) {
    System.out.println(get.getNom());
}
```

Écrivez la méthode `addIfPossible` qui produira un message d'avertissement si l'ajout n'est pas possible.

5 Groupes et étudiants

- une personne a un nom et un prénom ;
- un étudiant est une personne, qui est inscrite dans un groupe un seul

1. Ajoutez les classes `Person`, `Student` et modifiez `Group` pour avoir la liste de ses membres.
2. Les étudiants sont créés (en tant qu'entités administratives) par la méthode `registerStudent` de `School`, qui les enregistre dans les groupes.

```
/**
  Enregistre un étudiant et l'affecte à un groupe.
  @param firstName
  @param lastName
  @param group
  @throws   StudentRegistrationException  si déjà dans un groupe
*/
```

3. Écrivez du code de test qui crée 2 groupes dans une école, y enregistre deux étudiants, et fait afficher les groupes avec leurs membres.

6 Regroupement des exceptions

Modifier les deux classes d'exception pour qu'elles dérivent d'une classe plus générale `RegistrationException`.

Construisez un exemple d'utilisation qui en fait usage.

7 Groupes et profs

Vous ajouterez maintenant les enseignants :

- un enseignant est une personne, qui enseigne à des groupes (ensemble vide initialement).
- les enseignants sont enregistrés par l'école.

Dans la classe `School`, il faudra

- un attribut pour mémoriser la liste des enseignants,
- une méthode `registerTeacher(firstName, lastName)` qui enregistrera, et retournera un nouvel enseignant,
- des méthodes sur les groupes : `addTeacher`, `allTeachers` et `removeTeacher()`

À faire :

- écrire du code de test qui met en oeuvre ces méthodes,
- les écrire.